

RISE IN × STELLAR PRO HACKATHON 2026 • GENESIS TRACK

Jüri karşısında kanıt defteri

Dört iddia, dördünün de terminalden bir dakikada doğrulanabilir karşılığı.
Sorulduğunda ne söyleyeceğin, hangi komutu çalıştıracacağın, ve sormadan
önce senin söylemen gereken şeyler.

payper.live

Stellar testnet

Can Sarıhan • Berk Çiçek

1 · DeFi getiri protokolü — DeFindex

JÜRİ SORARSA İLK CÜMLEN

"Hazinemiz bir DeFindex vault'u. Söylememize gerek yok, kanıtı zincirde: vault'umuzun çalıştırdığı wasm hash'i, DeFindex'in kendi deposunda yayımladığı hash ile **birebir aynı**."

Çalıştıracağın komut

```
$ curl -s https://api.stellar.expert/explorer/testnet/contract/CBXHELM6...0DCP \
  | python3 -c "import json,sys; print(json.load(sys.stdin)['wasm'])"

f345228dca59c6605789620e9ec62ff4847a0927c33dac7581a955fe746016be

$ curl -s https://raw.githubusercontent.com/paltalabs/defindex/main/public/testnet.contracts.json \
  | python3 -c "import json,sys; print(json.load(sys.stdin)['hashes']['defindex_vault'])"

f345228dca59c6605789620e9ec62ff4847a0927c33dac7581a955fe746016be
```

Aynı olmaları tesadüf değil: vault'u **onların factory'si** deploy etti (CDSCWE46...4A32), o yüzden kodu da onların. DeFindex vault'u olduğunu söyleyip başka kod çalıştıran bir kontrat bu karşılaştırmada düşerdi. İlk çalıştırılacak kontrol bu.

Ardından gösterilecekler

stellar contract invoke okuma için bile ücreti ödeyecek bir kaynak hesap ister: `--source payper-admin --network testnet`. Tam komutlar README'nin **Proving it, not describing it** bölümünde.

Komut	Ne gösteriyor
invoke ... -- name	DeFindex-Vault-payper Treasury — vault bir token, adı factory tarafından verildi
invoke ... -- balance -- id <adaptör>	Hazinemizin vault payı: 264.979.712 (toplam arzın %99,9996'sı)
invoke ... -- total_supply	264.980.712 — aradaki 1.000 vault'un minimum likidite kilidi
invoice -- treasury_assets	Fatura kontratının fiyatlarırken gördüğü tutar, adaptör üzerinden vault'tan okunuyor

Mekanizma — "DeFindex'e gönderiyoruz" değil

fund() adaptörü çağırıyor, adaptör vault'a deposit yapıp **pay** alıyor, ödeme de karşılık gelen payı yakıyor. Pozisyon payların kendisi. Test ettik:

```
top-up : 2000.00 TRY → 40.7921817 USDC [completed] 9.6s
çekim  : 40.7921817 USDC → 1980.09 TRY [completed] +5.5s tx 644b32cf...
```

SORMADAN ÖNCE SEN SÖYLE

Vault'a strateji bağlı değil, çünkü testnet'te bu varlık için strateji yok — yani vault parayı tutuyor ama kazanmıyor. Adaptör ölçemediği bir oranı raporlamayı **reddedi**yor ve teklif failback etiketiyle geliyor. Arkasında pozisyon olmayan bir sayıyı "canlı" diye göstermektense bunu tercih ettik. Bu, entegrasyonun dürüst yarısı — ve koddaki böyle yazıyor: `panic_with_error!(&env, Error::NoYieldYet)`.

2 · Anchor

JÜRİ SORARSA İLK CÜMLEN

"Anchor hakkında kodumuzda yazılı tek şey alan adı. Alan adı, varlık, kur, uç noktalar — hepsi sayfa yüklenirken anchor'ın kendi stellar.toml'undan okunuyor. Başka bir anchor'a geçmek tek satırlık bir değişiklik; bir API yerine standart konuşmanın bütün sebebi bu."

Ekranda göstereceğin sıra

1. **Üstteki dört kutu** — bunlar keşfedildi, yazılmadı. Yanındaki "Check it yourself" şeridinden stellar.toml komutunu kopyala, terminalde çalıştır, ekrandakiyle karşılaştı.
2. **Akışı çalıştır.** Sağdaki terminal gerçek isteklerle doluyor — metot, yol, durum kodu, zaman damgası. SEP-1 → SEP-10 → SEP-12 → SEP-38 → SEP-6 sırayla geçiyor.
3. **Sonuçtaki transferler.** Her biri anchor'ın kendi işlem id'sini ve ürettiği Stellar işlemine giden linki taşıyor. İddia ile makbuz aynı satırda.
4. **Limit aşılsa** tutar transhara bölünüyor — 3.030 ₺ → iki kez 1.515 ₺, ikisi de completed.

Terminalden doğrulama

```
curl -s https://tr-mock-anchor.fly.dev/.well-known/stellar.toml
```

```
curl -s https://tr-mock-anchor.fly.dev/sep6/info
```

```
curl -s 'https://tr-mock-anchor.fly.dev/sep38/prices?sell_asset=stellar:USDC:GBBD47IF6LWK7P7MDEVSCWR7DPUVV3NY3DTQEVFL4NAT4AQH3ZLLFLA5&sell_amount=100'
```

EN GÜÇLÜ KANIT BİR HATADAN GELİYOR

Anchor, eşleştireceği memo tipini söylüyor. Biz MEMO_ID istenen yere MEMO_TEXT gönderiyorduk; ödeme hazinesine ulaşıyor ama izleyici tanıımıyordu ve işlem sonsuza kadar pending_user_transfer_start'ta kalıyordu. **Simüle edilmiş bir kanal bunu umursamazdı.** Düzelttikten sonra 9,6 saniyede completed. Bu hikâyeyi anlat — kanalın gerçek olduğunun en iyi kanıtı.

3 · Passkey

JÜRİ SORARSA İLK CÜMLEN

"Passkey **kimin sorduğunu** kanıtlar. Soroban XDR'ı tek başına imzalayamaz — WebAuthn kendi doğrulayıcı verisini kendi anahtar tipiyle imzalar. İkisini doğru köprülemek, secp256r1 imzalarını zincirde doğrulayan bir Soroban akıllı hesabı demek; o yol haritasında, bu derlemede değil."

Elimizde olan

Parça	Durum
WebAuthn töreni	İki yarısı da: register-options → register, login-options → login
Tekrar koruması	Cüzdan akışının kullandığı aynı tek kullanımlık challenge deposu; bir assertion tekrar oynatılmıyor
Sayaç kontrolü	Yükselen imza sayacı — klonlanmış bir doğrulayıcı böyle yakalanır
Adres türetimi	Kimlik bilgisi + sunucu sırrı → SHA-256 → Ed25519 anahtarı
Cüzdan bağlama	Passkey ile girip sonradan tarayıcı cüzdanı kanıtlama; kanıt girişle aynı challenge ve imza

Çalıştıracığın komut

```
curl -s https://payper.live/api/auth/passkey
```

Türetimin deterministik olduğu, deponun kopyasından:

```
PASSKEY_WALLET_SECRET=demo npx tsx -e "import {keypairForCredential} from './src/lib/auth/passkey'; for (const c of ['cred-abc', 'cred-abc', 'cred-xyz']) console.log(c, keypairForCredential(c).publicKey())"
cred-abc GACC2HRYKYLUQKJ3WJ5CZGU4GNLNRHZNMODRL5Z7NPQFAT06SWCH2G0L
cred-abc GACC2HRYKYLUQKJ3WJ5CZGU4GNLNRHZNMODRL5Z7NPQFAT06SWCH2G0L
cred-xyz GBHKX4UKZKZNRVUUPEYIFG6KFW2EZHLW0TA2GXVBK7D33T64AI2NE3NE
```

Aynı passkey hep aynı adrese varıyor, seed phrase yok, kullanıcının saklaması gereken hiçbir şey yok.

SORMADAN ÖNCE SEN SÖYLE

İmzalayan anahtar **sunucuda** duruyor. Cüzdan ekranı bunu aynen bu cümleyle yazıyor. Çıkış yolu da orada: passkey ile gir, sonra tarayıcı cüzdanını bağla — o andan itibaren oturumun senin tuttuğun bir anahtarı olur.

4 · E-faturanın zincir üstündeki hâli

JÜRİ "TOKENİZE ETTİNİZ Mİ" DERSE

"Alacak zincir üstünde **bölünebilir, devredilebilir ve hamiline ödenen** bir hak olarak duruyor. Fonlayıcı payını vadeden önce başkasına satabiliyor, `repay()` vadede o an kimin elindeyse ona ödüyor. Aynı bir token kontratı basmadık — hak, fatura kontratının kendi depolamasında; ama tokenizasyonun verdiği özelliklerin hepsi burada."

Gerçekten olan

Zincirde ne var	Nasıl gösterilir
ETTN'in SHA-256'sı, kalıcı depolamada, TTL maksimuma çekilmiş	<code>is_ettn_available --ettn_hash ...</code> → ikinci finansmanı imkânsız kılan anahtar
Belgenin SHA-256'sı	<code>get_invoice --invoice_id 4</code> → belge sonradan değişmediği kanıtlanabilir
Fonlayıcı başına bölünebilir talep	<code>funders_of --invoice_id 2</code> → adres ve tutar
Devredilebilirlik	<code>transfer_claim</code> → sahibi imzalar, hak yer değiştirir, toplam değişmez
Vadede oransal dağıtım	<code>repay()</code> defteri gezip her fonlayıcıya payını gönderiyor

Talep hakları devredilebiliyor

Vadeyi bekleyemeyen bir fonlayıcı payını bekleyebilecek birine satıyor ve `repay()` vadede **o an kimin elindeyse ona** ödüyor. Bizden izin istenmiyor, alıcıya sorulmuyor — kontrat payı taşıyor çünkü sahibi imzaladı.

```
stellar contract invoke --id CC76VPEQA6SCHUOWL62RS04EVWWZXACNUYJENREZSA4BGZIOJQ3EJ4R --source payper-admin --network testnet --claim_of --invoice_id 2 --holder <G...>
```

```
stellar contract invoke --id CC76VPEQA6SCHUOWL62RS04EVWWZXACNUYJENREZSA4BGZIOJQ3EJ4R --source payper-funder --network testnet --send=yes --transfer_claim --invoice_id 2 --from <G...> --to <G...> --amount 133680000
```

Gerçekten çalışmış bir devir, 2 numaralı faturada:

	fonlayıcı A	fonlayıcı B	toplam
öncesi	26,7360	26,7360	53,4720
sonrası	13,3680	40,1040	53,4720

tx 2ad7750d2129bc5f0bcc0b62e6538d5685f9c90bc9fa915de7fce912ec40e86a

Defterdeki toplam, kontratın vadede ödeyeceği tutardır; o yüzden her devirde sabit kalmak zorunda.

`transferring_never_changes_the_total` bunu sabitleyen test, `repayment_follows_the_claim` ise ödemeyi devralanın aldığı kanıtlıyor. 29 kontrat testinin yedisi bu özellik için.

YİNE DE BASMADIĞIMIZ ŞEY

Fatura başına ayrı bir token basmıyoruz. SEP-41 kontratı ya da NFT yok; talep hakkı ayrı bir varlıkta değil, fatura kontratının kendi depolamasında duruyor. Bu bilinçli: fatura başına ikinci bir varlık, doğru tutulması gereken ikinci bir şey demek. Önemli olan üç özellik — **bölünebilirlik, devredilebilirlik ve hamiline ödeme** — üçü de artık yerinde.

Sana sorulacak dört zor soru

"Bunu blockchain olmadan da yapamaz mıydınız?"

Kabul et, sonra asıl cevabı ver. ETTN tekiliği, fiyatlama, oransal dağıtım, banka transferi — hepsi bir veritabanıyla yapılır. Türkiye'de KKB'nin Fatura Kayıt Sistemi zaten çift finansmanı engelliyor. Zincirin yaptığı üç şey var: **(1)** fonlayıcı kitlesini açıyor — faktoring şirketinin tavanı bilançosudur, burada USDC tutan herkes elli dolarla fonlayabiliyor; **(2)** fiyat bize sorulmadan doğrulanabiliyor; **(3)** vadede ödeyen biz değiliz, repay() biz batsak da çalışıyor. UBL ayırıştırma, KYC ve banka bacağı klasik yazılım — oraya zincir sokmadık.

"Bir çekin birden fazla sahibi olamaz"

Çek için haklılar, fatura için değil. Çek kambiyo senedidir, kısmi ciro geçersizdir. Biz çek değil **e-fatura alacağı** finanse ediyoruz; adı alacakta kısmi temlik geçerlidir ve para alacağı bölünebilir. Üstelik temlikin borçluya karşı hüküm doğurması bildirim gerektirir — bizim acknowledge() adımı tam buna oturuyor. Ayrıca kontratımız mülkiyet devretmiyor, ödeme üzerinde oransal katılım yaratıyor; bu daha da rahat bir zemin. (Avukata teyit ettirin.)

"Sahte fatura kesip finanse ettiremez miyim?"

Sahte fatura kesilebilir, biz engellemiyoruz — yapıyı doğruluyoruz, varlığı değil. Kapi acknowledge(): faturada yazılı adres imzalamak zorunda, o yüzden gerçek bir firmaya kesilen sahte fatura asla fonlanmıyor. Durdurmadığı şey muvazaa — iki cüzdanlı tek kişi. Buna karşı bugünkü sürünme anchor'ın SEP-12 KYC'si, tedarikçiye rücu ve whitelist eşliği; hepsi ekonomik ve hukuki, hiçbir kriptografik. Kapatacak iki şey yol haritasında: GİB doğrulaması ve VKN ↔ adres bağı.

"Kur riskini kim taşıyor?"

Henüz modellenmedi, söyleyin. repay() sabit face_usdc çekiyor, alıcı ise sabit lira borçlu; doksan günde ikisi uyuşmayı bırakıyor. Kur primi (723 bps, canlı ölçülüyor) riski **fiyatlıyor, atamıyor**. Üretimde iki dürüst çözüm var: ya alıcı lira cinsinden yükümlü kalır ve platform açığı hedge eder, ya da fonlayıcının alacağı liraya endekslenir. "Çözdük" dersiniz ilk takip sorusunda düşersiniz.

Üç dakikalık demo sırası

Sn	Ne yapıyorsun	Ne söylüyorsun
0–20	Ana sayfa, canlı rakamlar	Sermaye Türk bankalarının bilançosunda kilitli; biz o kapıyı açıyoruz
20–40	Fatura yükle → kopya dosyayı da yükle	Aynı ETTN, farklı belge. Kontrat reddediyor — politika değil, değişmez kural
40–60	Alıcı onayı	Alacağı bir yabancı için gerçek kılan tek imza
60–90	İskonto kırılımı	Dört bileşen, kur Live, 29 günlük pencere ve sapma ekranda
90–120	Sesle ödeme — telefonla QR, tonları çal	Kamerasız, eşleşmesiz, veri bağlantısız. Ses istek taşıyor, yetki değil
120–150	Anchor akışını çalıştır	Terminal gerçek isteklerle doluyor; her transferin Stellar tx linki var
150–180	Kapanış / temerrüt şelalesi	İki bitiş yolu da aynı kontratta; erken temerrüt zincirde reddediliyor

KAPANIŞ CÜMLEN

"Buradaki her sayı şu an zincirden okundu. Hangisine şüpheyle bakıyorsanız komutu ekranda, kopyalayıp kendiniz çalıştırabilirsiniz. Bizim iddiamız değil, sizin doğrulamanız."